Superscalar Organization

Chester Rebeiro

Indian Institute of Technology Madras

chester@cse.iitm.ac.in

Limitations of Scalar Pipeline

$$\text{CPU Performance} = \frac{1}{\text{instruction count}} \times \frac{\text{instructions}}{\text{cycle}} \times \frac{1}{\text{cycle time}} = \frac{\text{IPC} \times \text{frequency}}{\text{instruction count}}$$

For a scalar pipeline, this is bounded by 1

Frequency

(1) Deeper pipelines would mean,

- complexity of forwarding blocks and hazard detection increases.
- Delays due to pipeline buffers start becoming non-significant.
- Stalls due to branch mis-prediction become significant.

(2) Instructions / cycle bounded by 1

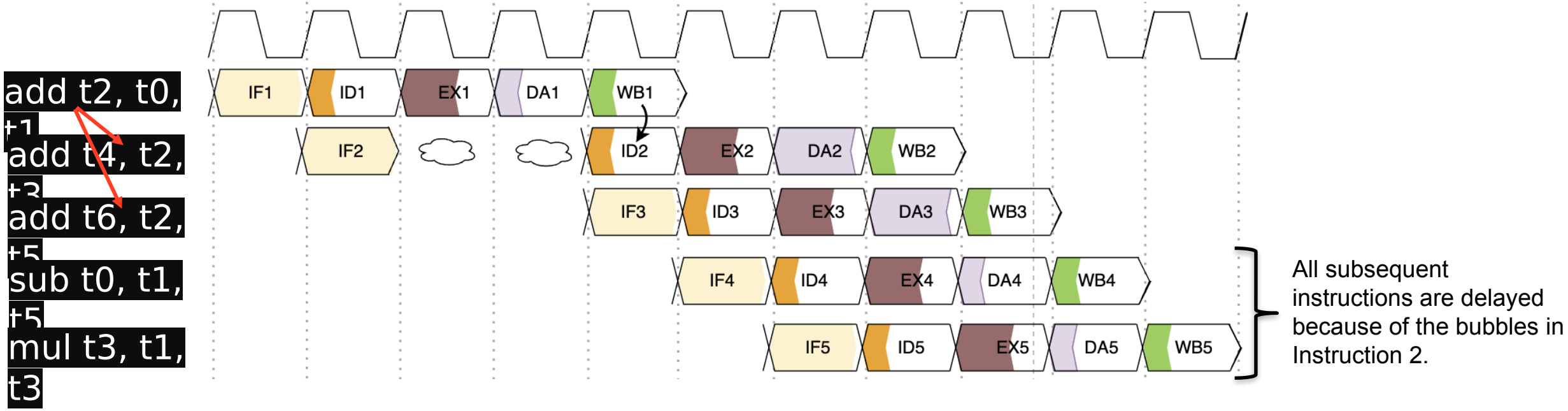
(3) Instructions can be considerably diverse, but the scalar pipeline works best with identical instructions.

- For example, MUL and DIV has considerably longer latencies compared to ADD and SUB. MUL and DIV ideally require multiple clock cycles, while ADD and SUB require a single clock cycle.
- However, in a scalar pipeline, all these instructions should execute identically (same number of delays or same clock cycles)

Limitations of Scalar Pipeline

(4) Performance lost due to rigid pipeline.

- Instructions flow in a lock-step fashion in the scalar pipeline. When an instruction stalls for N clock cycles, all subsequent instructions are delayed by N clock cycles.

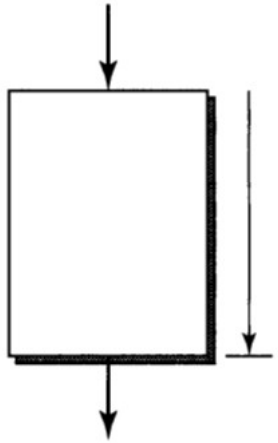


From Scalar to SuperScalar Pipelines

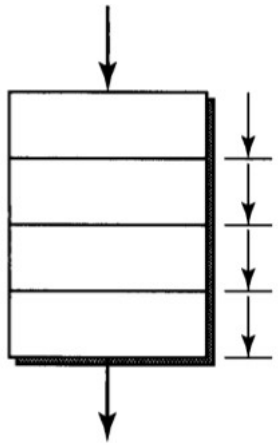
- Overcomes the limitations of Scalar Pipelines
  - (2) No longer bounded by 1 instruction / cycle. Possible to complete >1 instructions/cycle
  - (3) Can efficiently cater to diverse instructions
    - MUL, DIV, ADD, and SUB, do not have to take the same number of clock cycles or have the same delay
  - (4) Rigid pipeline
    - Permits instructions to execute out-of-order, if there are no dependencies. Thus, a stall in one instruction may not delay a subsequent instruction's execution.

Achieved by Diversified Parallel Pipeline

Parallel Pipelines

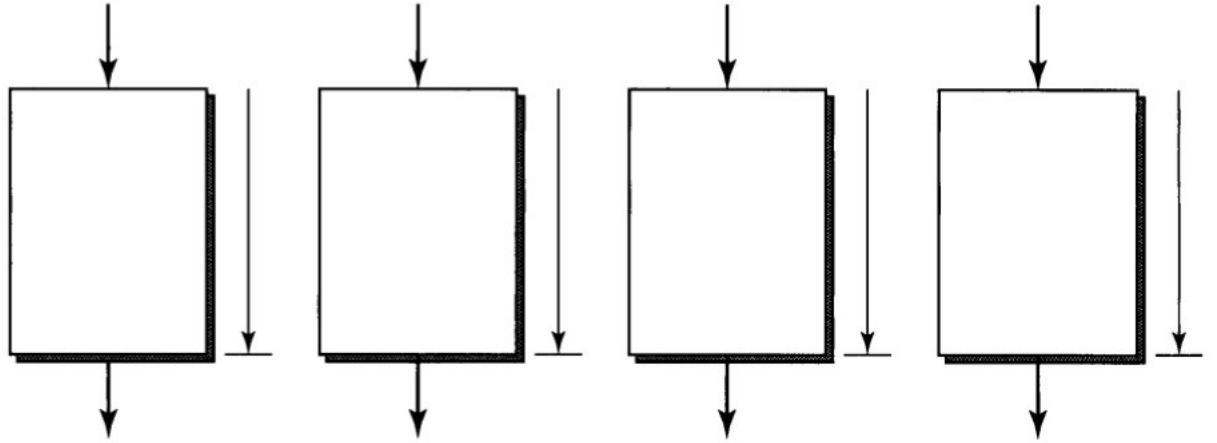


(a) No parallelism

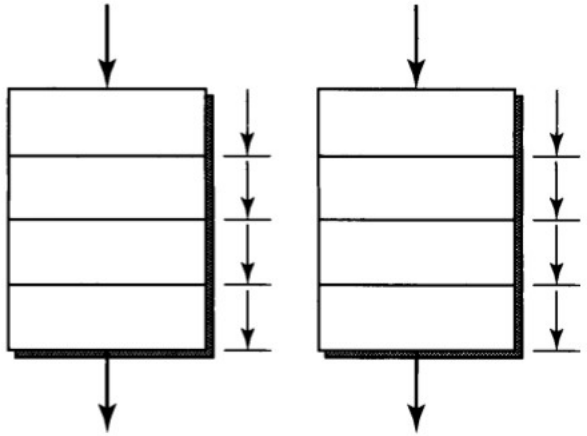


(b) Temporal parallelism

Regular pipeline



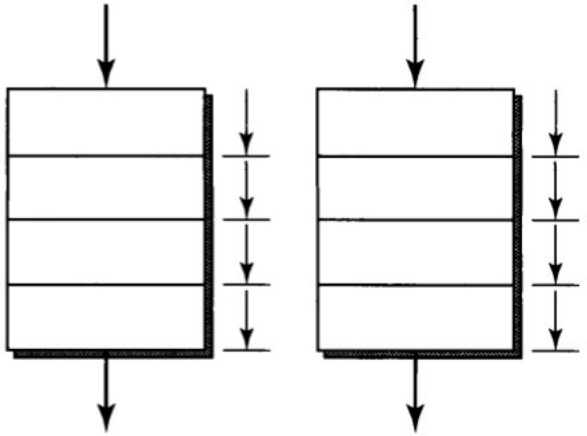
(c) Spatial parallelism



(d) Parallel pipeline

Combines Temporal and Spatial parallelism

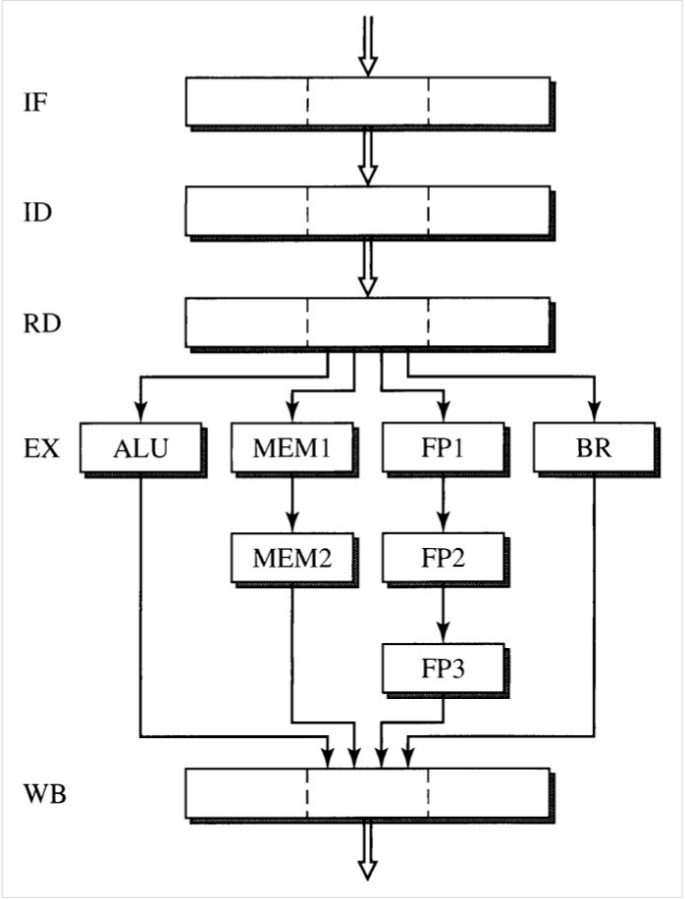
Diversified Pipeline



(d) Parallel pipeline

Parallel Pipelines do not cater to the diversity of instructions – eg. Mul, div, add, and sub have different hardware and latencies

Diversified execution pipes cater to different instructions. Each pipeline has different depth catering to different instructions.

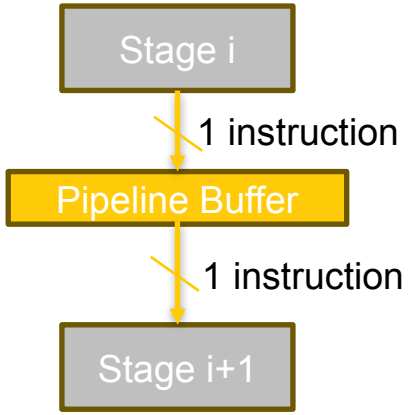


A diversified pipeline with 4 execution pipes

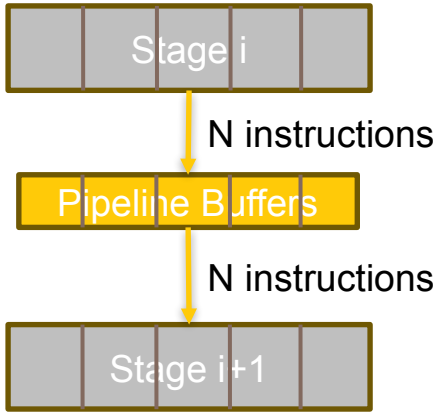
3-issue pipeline: Can issue 3 instructions simultaneously to each stage

Pipeline Buffers in Scalar and Parallel Pipelines

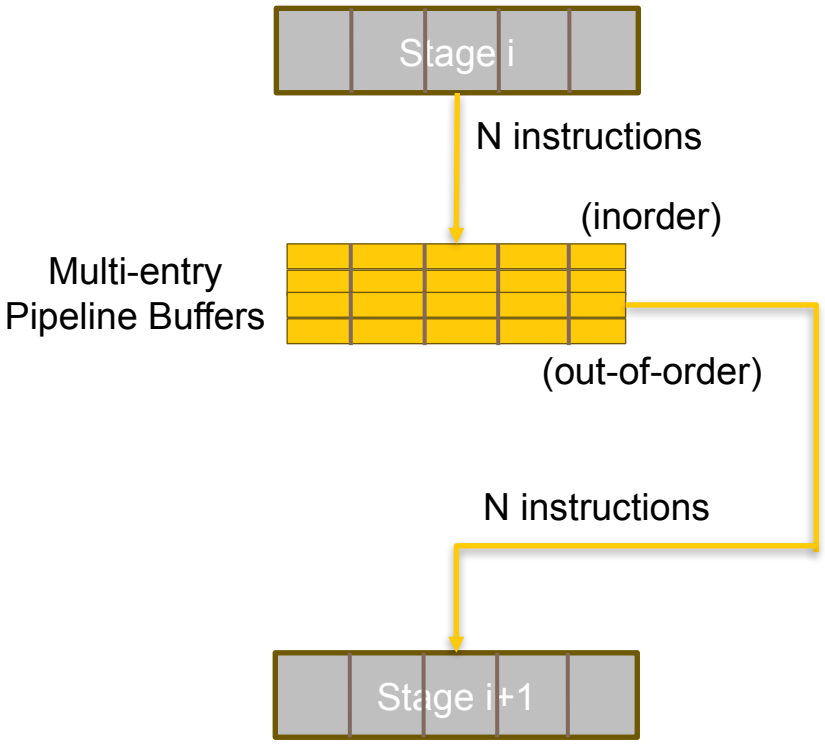
Scalar Pipelines make use of **single pipeline buffers** between stages. These buffers hold single-entry buffers. That latch the result and control signals produced in stage i. These are used in stage i+1.



Parallel Pipelines make use of **multi-entry pipeline buffers** between stages. Multiple instructions can be latched into buffers. The latch holds result and control signals produced in stage i. These are used in stage i+1.



Multi-entry buffer with reorder

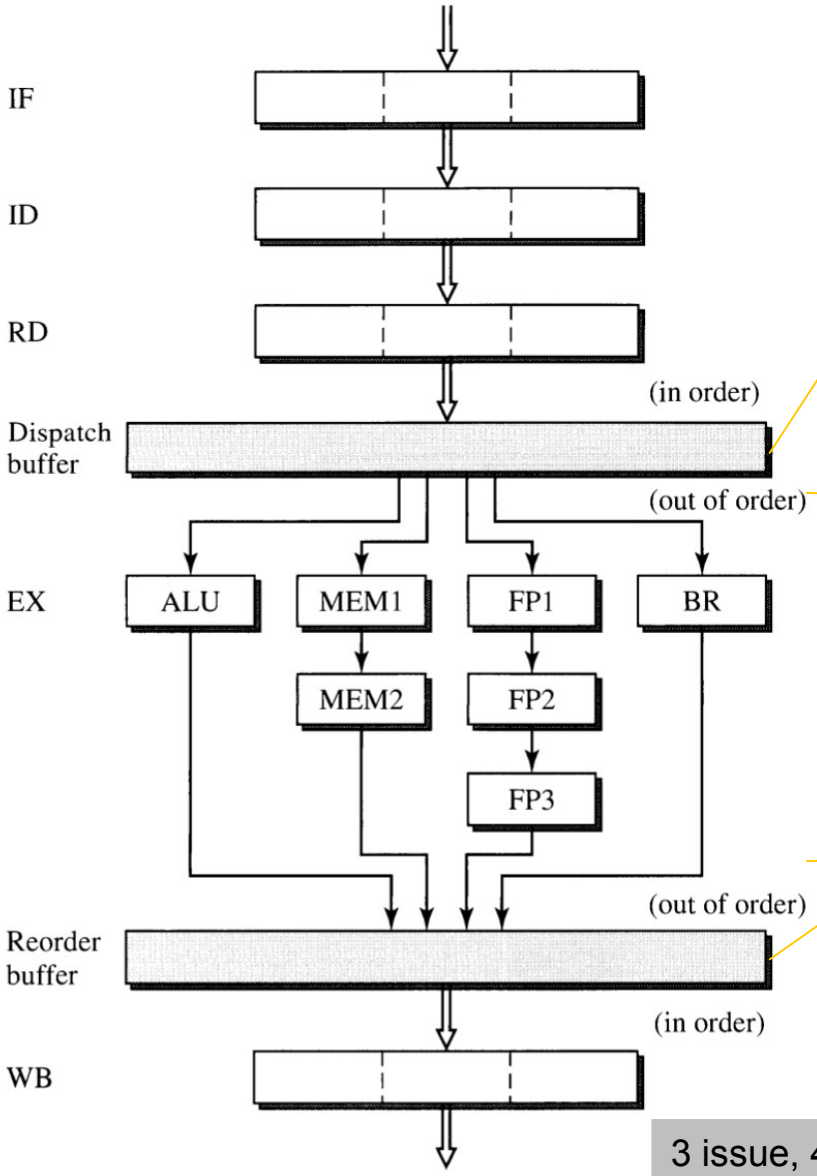


Multiple decoded instructions latched into each multi-entry buffer in every clock cycle by Stage I (in-order)

Multiple instructions read from any entry in the multi-entry buffer in Stage i+1. Not necessarily in the program order.

While in the buffer, instructions can be updated or modified. For example, by forwarding paths.

Multi-entry buffer in a dynamic pipeline



Dispatch Buffer. Loaded with decoded instructions according to program order.

(EX) Execution Stage. 4 pipelines. Each pipeline caters to a class of instructions, has different cycle requirement. Instructions are scheduled out-of-order, based on the availability of a pipeline.

Reorder Buffer. Commits the results in the order of the program to ensure correctness. Instructions which completed early, are buffered until other results are available. Multiple results can be committed in a single clock cycle.

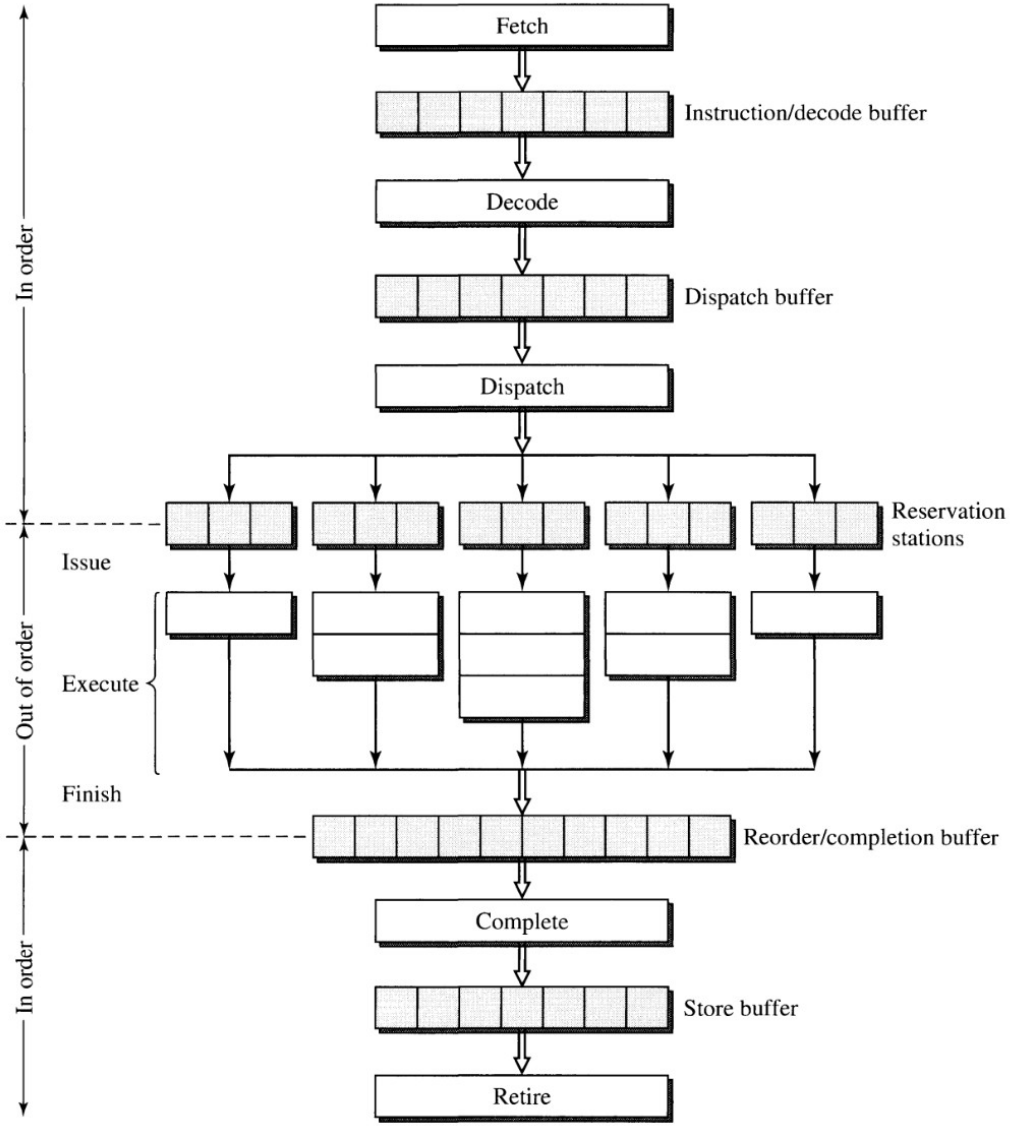
3 issue, 4 execution pipelines

Superscalar Pipeline Overview

TEM: An Example of a Superscalar Pipeline

6 stages: Fetch, Decode, Dispatch, Execute, Complete, Retire

Dispatch stage: Associate instruction with a functional unit (like ALU/FPU etc)
Issuing buffer: used to initiate execution
Execute stage can have multiple pipelines
Complete stage reorders instruction results to ensure the results are in-order



Instruction Fetch Stage

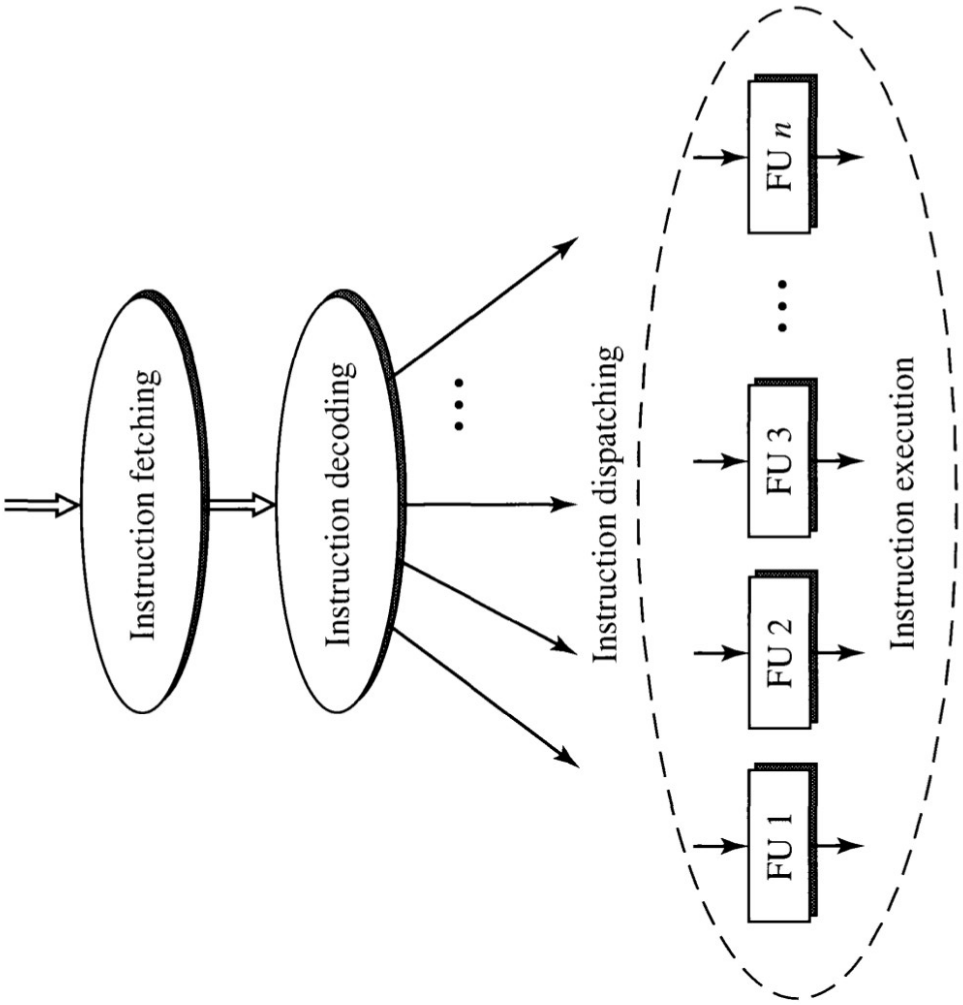
- Unlike scalar pipeline, fetch more than one instruction from Instruction (I)-cache in every clock cycle.
- If s-issue pipeline, IF stage will fetch s-instructions from I-cache
  - If PC points to address A, then the instruction at A and subsequent s-1 instructions are fetched. This is called the **Fetch Group**.
  - ideally multiple instructions from the same cache line is fetched.

Decode Stage

- Unlike, scalar pipeline, superscalar decode stage only decodes the instruction. It does not read the register contents.
- Multiple instructions are simultaneously decoded and dependencies between instructions identified.
  - Early identification of dependencies helps parallelize instructions better
- On RISC machines, the decode stage mainly contains a large number of comparators, multi-ported register files, etc.
- On CISC machines, the variable length instructions make it even more complex. On Intel processors, instructions converted to micro-ops.

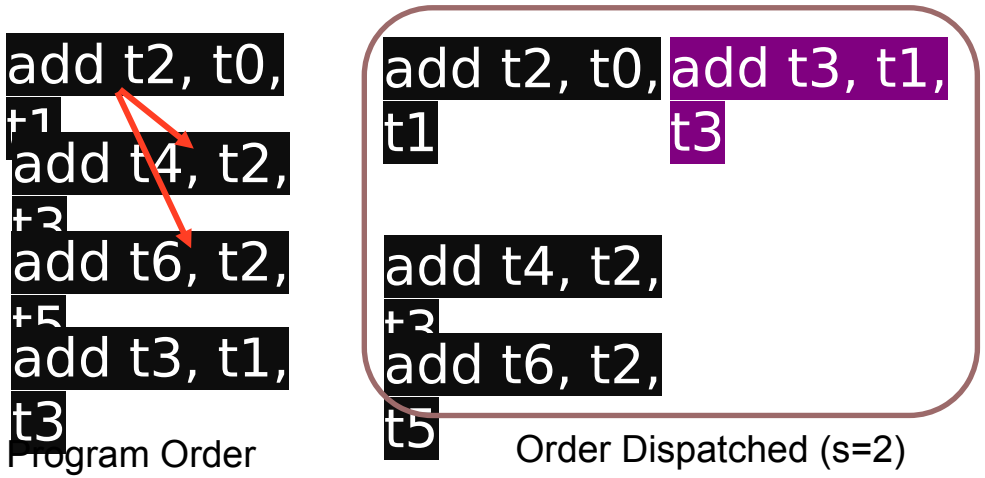
Fetch / Decode unit in Intel P6 processors

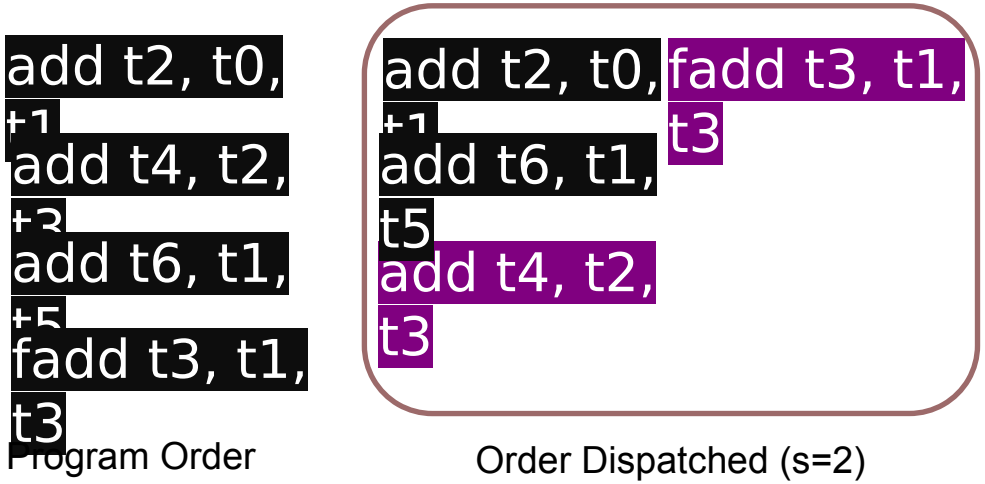
Instruction Dispatch



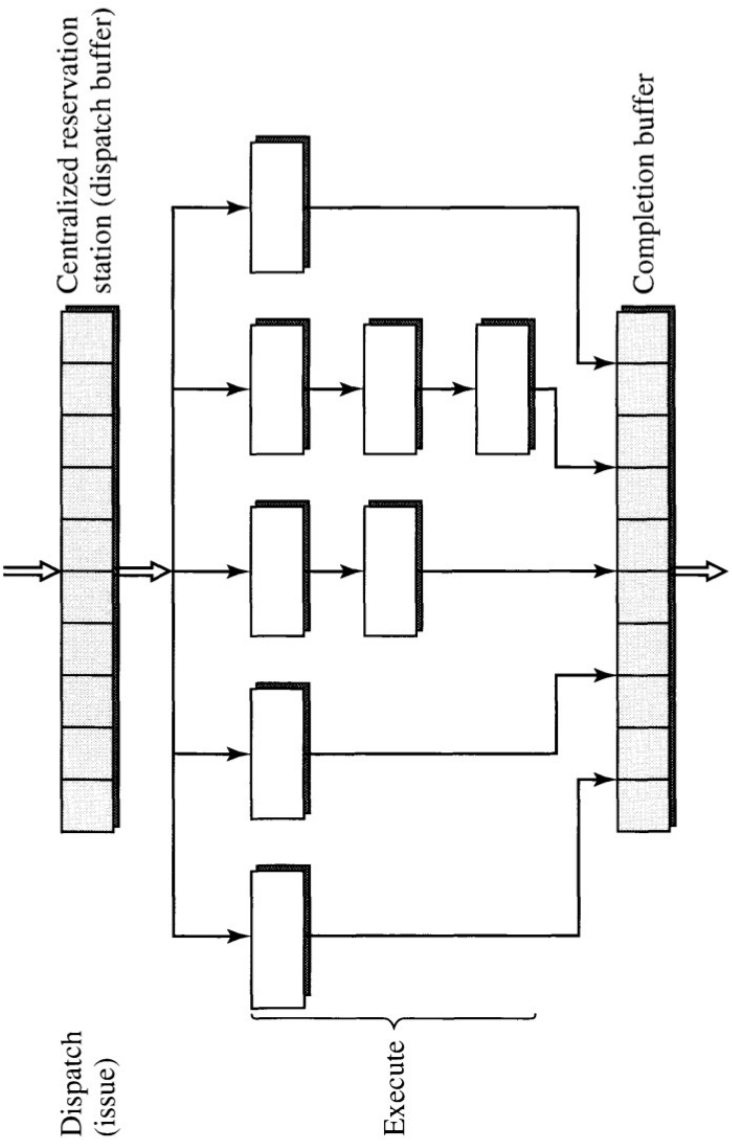
- Between decode and execute. This stage is required because of the diversified pipelines.
- Instructions are dispatched to a pipeline based on,
  - Type of instruction (eg. Load/store dispatched to the memory operations pipeline while add/sub dispatched to the ALU)
  - Availability of operands. All operands needed for an instruction should be available.
  - Availability of a pipeline (eg. some processors have 2 ALUs. Instructions are dispatched to an ALU based on the load on that ALU)

Instruction Dispatching



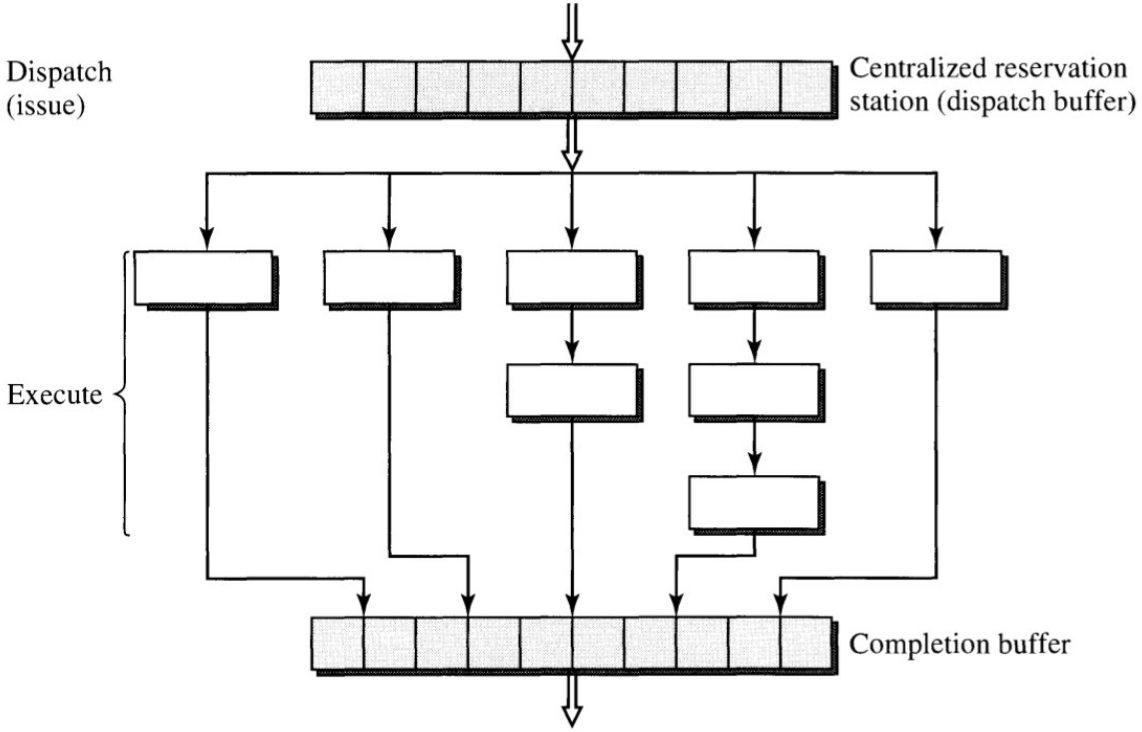


Dispatch Buffer

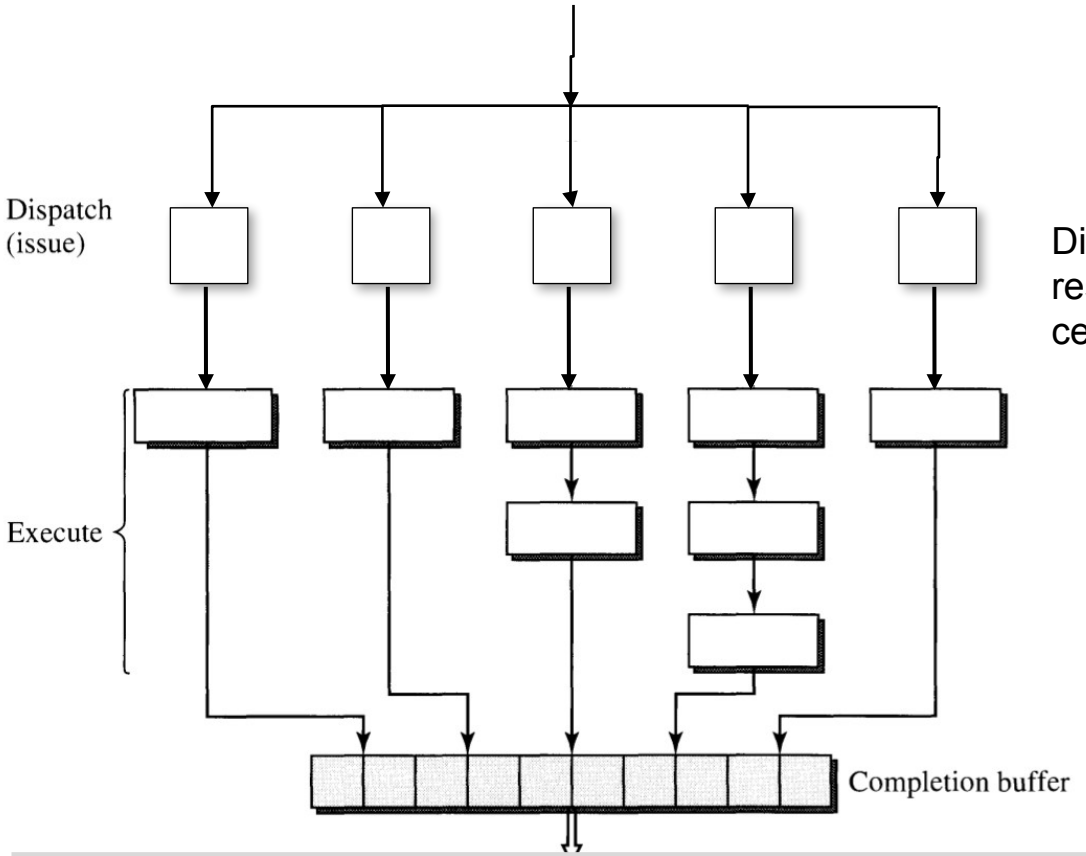


- A [illegible] d for an instruction should be available for an instruction to be dispatched and the pipeline must be available.
- Until then, instruction is stored in the dispatch buffer, other instructions which are independent and whose operands are ready can be dispatched.
- Thus, instructions may be dispatched out-of-order, as soon as its operands are available.
- s instructions can be dispatched simultaneously (s is the issue)
- Dispatch Buffer also called Reservation Station

Dispatch Buffer Centralized or Distributed



Centralized Reservation Station:
Hold all instructions regardless of type
More complexity (multi-ported buffers)
better utilization or resources



Distributed
reservation
centers

Distributed Reservation Station:
Simple single ported multi-entry buffers that store instructions corresponding to a specific functional unit
Utilization of buffers may not be uniform, causing stalls

Execution Stage

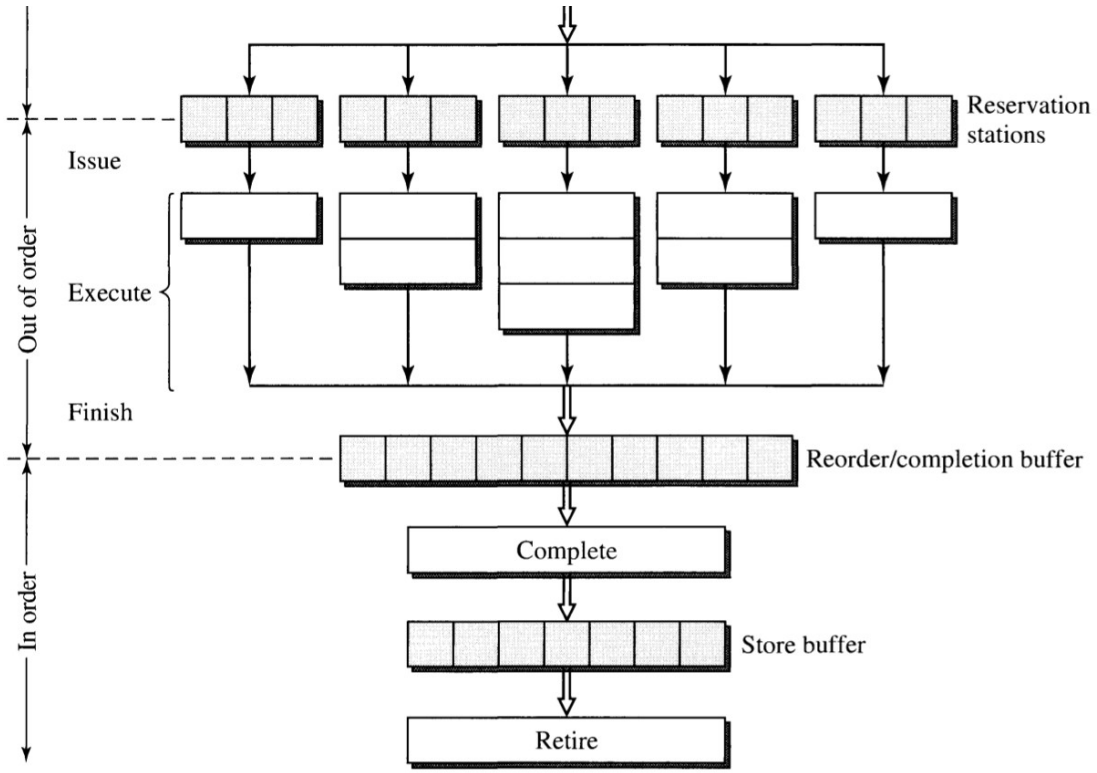
- Can have multiple similar functional units, for example two independent pipes for ALUs
- Can perform compound arithmetic operations, like (AxB) + C
- Support for Single Instruction Multiple Data types

What is the right composition of functional units?
4-2-4 rule is one example: For every 4 ALU units, have 2 Branch units, 4 memory units (in practice, have fewer memory units due to memory bottlenecks)

How much of parallelism vs pipelining is appropriate?

Instruction Completion and Retiring

- Instruction is 'completed' when it finishes execution
  - Such instructions are stored in a completion buffer
  - Results stored in temporary registers
- Store Instruction are 'retired' when the D-cache is updated.



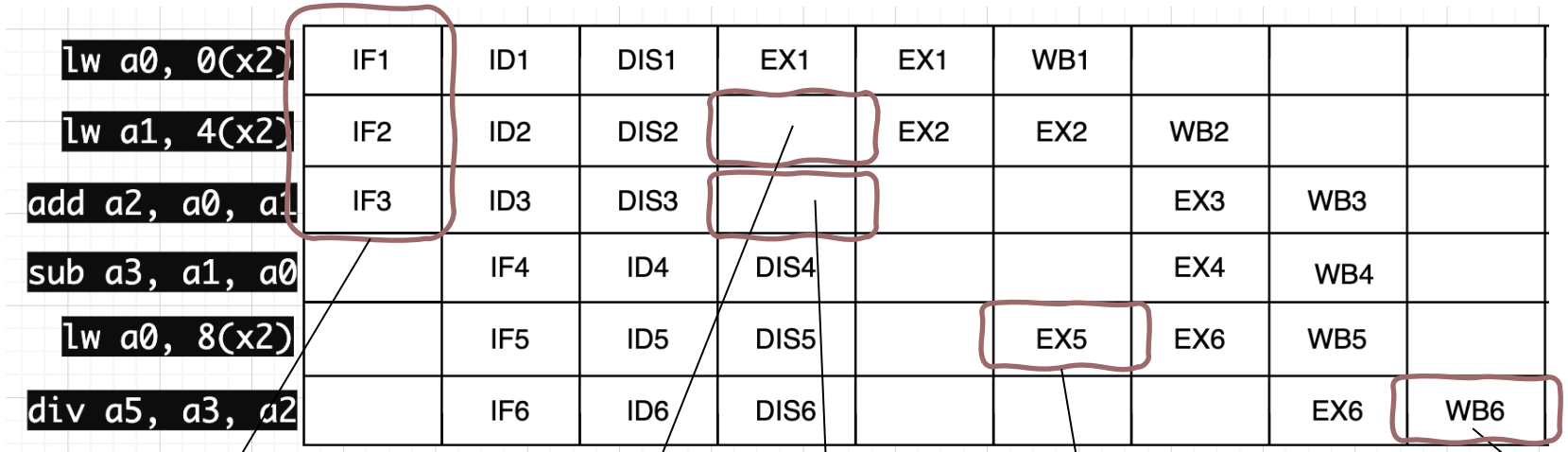
An example of Superscalar instruction flow

- Consider a 3-issue superscalar processor with stages: Instruction Fetch, Decode, Dispatch, Execute, and Writeback
- There is one 2-stage load/store unit, 1-pipeline stage ALU, and 1-pipeline stage mul/div unit
- Consider the following instructions.

```
lw a4, 0(x2)
lw a1, 4(x2)
add a2, a4, a1
sub a3, a1, a0
lw a0, 8(x2)
div a5, a3, a2
```

- Outline the flow of instructions through this CPU.

Instruction flow in the CPU



`lw a0, 0(x2)`	IF1	ID1	DIS1	EX1	EX1	WB1			
`lw a1, 4(x2)`	IF2	ID2	DIS2		EX2	EX2	WB2		
`add a2, a0, a1`	IF3	ID3	DIS3				EX3	WB3	
`sub a3, a1, a0`		IF4	ID4	DIS4			EX4	WB4	
`lw a0, 8(x2)`		IF5	ID5	DIS5		EX5	EX6	WB5	
`div a5, a3, a2`		IF6	ID6	DIS6				EX6	WB6

3 issue, therefore at-most 3 instructions move from one stage to the next in each clock cycle

Can't move Instr 2 to Execute in this cycle, because there is only one load/store pipeline

Can't move Instr 3 to Execute in this cycle, because of dependencies with Instr 1 and Instr 2.

Executing Instr 5 out-of-order because it does not have any dependencies. We can't execute it in previous clock cycle, because of EX2 occupies the load/store pipeline

Writebacks always happen inorder

Instruction flow in the CPU (with an exception)

